Balanced Search Trees

Balanced Search Trees

As we saw in the previous section, BSTs give us time for insertion, deletion and search of $O(height)$. And the average height of a randomly constructed BST is $O(\log n)$, but the worst case is $O(n)$.

But if we could find some way to balance the tree, the height would be $O(\log n)$, which would make all of our operations very efficient.

Content

- Balance
- AVL Trees
- AVL Insert
- AVL Delete

Balanced Search Trees

A balanced tree has some concept of keeping a similar number of nodes in each branch of every subtree.

There are many balanced search tree data structures, including:

- Red-Black Trees
- AVL Trees
- 2-3 Trees
- B-Trees
- Splay Trees
- etc.

Since these trees are balanced, they all give $O(\log n)$ asymptotic time for all operations. However, different trees make different trade-offs in the way that they define and maintain balance and in the general structure of the tree. Some are faster when inserting and deleting (e.g. red-black trees), some are slightly more compact and therefore better when searching (e.g. AVL Trees), and some are designed to optimise memory access (e.g. B-Trees).

Red-black and AVL trees are both variants of *binary* search trees, and if fact they work exactly like a regular BST when searching. The only difference is that when inserting and deleting they check for balance and modify the shape of the tree if necessary.

We will take a closer look at AVL trees.

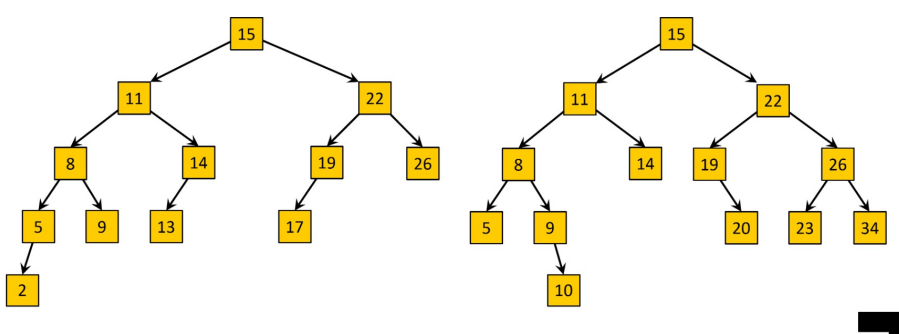
AVL Trees

An AVL-tree is a self-balancing BST, named after Adelson-Velsky and Landis (1962).

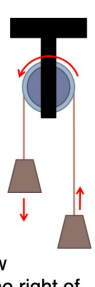
In addition to the order property of a BST, an AVL also enforces a balance property.

> ⚠ ***AVL Balance Property***
>
> *The heights of the left and right child of every node in an AVL tree differ by at most 1*

For example, in the image below, the tree on the left is balanced, and the tree on the right is not balanced. In the tree on the right, node 11's left child has a height of 2, whereas the right child has a height of 0, so the difference is 2. (Note that, when working with AVL trees, we say that the height of an empty tree is -1).



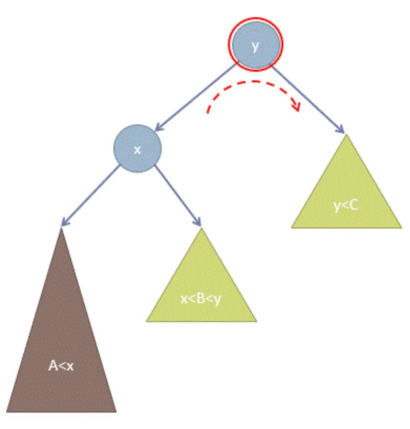
When a node is unbalanced (which may only happen after an insertion or deletion), there will be at least one node where the different in height of the left and right sub-trees is exactly 2. To rebalance the tree a *rotation* is performed. You can think of a rotation as being like two weights on a pulley, where the pulley is where the unbalanced node is and the weights are the two sub-trees.



Single Rotation

The following animation shows a single rotation to the right. The diagram only shows the unbalanced subtree (this could be anywhere in the tree) with the unbalanced node as the root. The triangles represent additional sub-trees with the brown subtree having a height that is one more than the yellow subtrees. So you can see that the height of the left subtree of y, with root x, must be one more than the right of the right subtee. After the rotation, you can see that the left and right sub-trees now have the same height.

Single Right Rotation

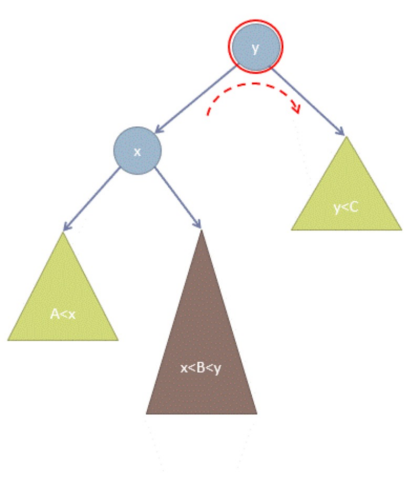


The important thing to note here is that to perform this rotation, only three pointers need to be changed.

- The right child of x becomes the left child of y.
- y becomes the new right child of x.
- And whatever reference originally pointed to y (it could have been a left or right child of another nor, or the root of the tree), now needs to point to x.

Single rotations don't always fix the balance problem. In the following single rotation, you can see that the result is still unbalanced.

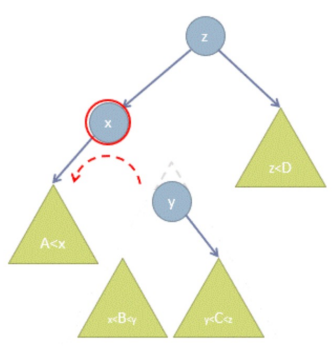
Single Rotation Fail



We fix this problem by performing a double rotation. We identify that a double rotation is needed by looking at the path from the unbalanced node that is too high. In the first example we see that longest path from y goes to the left and then left again (to the brown subtree). But in the second example the longest path is to the left and then right. When we have this left-right (or right-left) pattern for the path that is too high, we know that we need a double rotation.

To fix a left-right pattern (as above) we first do a left rotation at x, then to a right rotation at y. The next animation illustrates this (note that we have expanded the brown subtree for this example).

Double Rotation

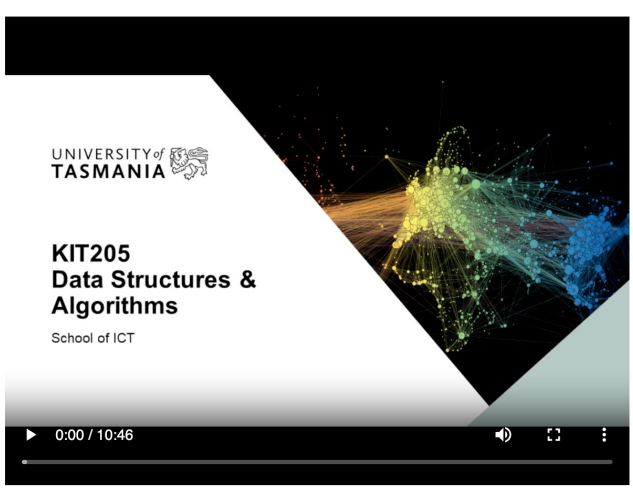


Now that we know how to fix balance, let's look at insertion and deletion.

Inserting into an AVL Tree

AVL insertions start by performing a normal BST insert operation, but after inserting you work back up from the point of insertion and rebalance at the first unbalanced node you find. You will find *at most* one unbalanced node.

▶ AVL Insert

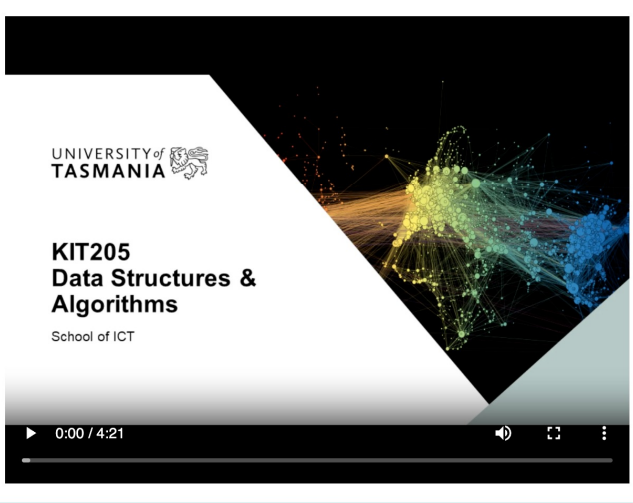


Deleting from an AVL Tree

As with insertion, deleting from an AVL tree starts the same way as deleting from a BST. And as with insertion, after deleting you would back up the tree and rebalance at any unbalanced nodes. Unlike insertion, you may find more than one unbalanced node as you work back up the tree.

The other complication is that you must start from the position that you actually removed a node. For example, iff the value to be deleted has two children, you will actually be removing the in-order successor node. So you would start at the in-order successor and work back up from there.

▶ AVL Delete



Summary

In this module we saw one example of a balanced search tree. By maintaining balance we are able to achieve $O(\log n)$ for all find, insert and delete operations. Furthermore, because it is an ordered data structure, we can also efficiently produce an ordered list.

Next we are going to look at some quite different data structures that improve on this performance for some operations.

> 📖 ***Further Reading***
>
> *The following website has some interactive animations for AVL trees, and many other data structure.*
>
> *Visualgo*
>
> *And see this page for discussion on other types of balance search trees.*
>
> *Balanced Search Trees*